CS112 Final Project
Naomi Arnold & Katie Casey
12/8/2025

Proxy Design

Overview

Our project implements an LLM-powered web proxy that translates web pages into multiple user-selected "styles," including **Simplified**, **Pirate Speak**, **Gen Z slang**, and **TL;DR summaries**. The proxy is fully implemented in **C**, leveraging the provided LLM proxy API to send extracted page content for transformation and receive rewritten output.

The proxy intercepts web traffic, extracts textual content, processes it through the LLM, and serves the transformed pages to the client's browser. Users can select a desired style through a simple interface, allowing dynamic, real-time content translation. The proxy works using the following features:

1. **Traffic Interception**
    - For **HTTP**, the proxy listens on a local port and intercepts requests from the browser. Upon receiving a request, the proxy fetches the page from the target server, parses the content, and performs style translation via the LLM. The transformed page is then returned to the browser.
    - For **HTTPS**, the proxy establishes a tunnel using the **CONNECT method**, allowing secure traffic to pass through. Currently, LLM-based translation is **not implemented for HTTPS pages** due to time and development constraints, not due to security concerns. The proxy successfully relays HTTPS traffic, and future work could extend LLM translation to these pages.
2. **Content Extraction**
    - The proxy parses HTML responses to isolate **main textual content**, excluding scripts, stylesheets, and large media elements.
    - This reduces the payload sent to the LLM, improving latency and avoiding unnecessary processing.
3. **Threading and Concurrency**
    - Each client request is handled in a separate thread, enabling **concurrent page retrieval and translation** without blocking other requests.
    - CSS, JavaScript, and media assets are fetched concurrently, further minimizing page load time.
4. **Caching and Preloading**
    - **Caching** stores translated pages in memory, allowing repeated requests for the same URL and style to be served instantly.
    - **Preloading** generates translations for popular pages in advance, reducing first-time latency when a user selects a style.
5. **HTTP vs HTTPS**
    - **HTTP pages**: Fully supported, including content transformation.
    - **HTTPS pages**: Supported for secure tunneling, but LLM-based transformations are **not yet implemented**.

The LLM-based features perform well for static, text-heavy websites, preserving semantic meaning while adjusting tone and style appropriately. However, on more complex sites with dynamic content or extensive media/formatting, the transformations are generally accurate but occasionally result in slightly erroneous formatting or modified visual context/interactive elements.

Performance Evaluation

Overall, browsing performance with and without the proxy was effectively the same, with only negligible differences within normal variation. Google.com (HTTPS) loads normally through the proxy, ~300ms as expected. Httpforever.com (HTTP) loads in ~400ms without the proxy, and ~420ms with the proxy (negligible variation).

The largest performance bottleneck in our system was the LLM processing latency. Initially, we sent most, if not all, of the webpage's raw HTML content to the LLM. This resulted in ~20 seconds of latency for small pages, and multiple minutes for larger or media-heavy pages. After implementing threading, caching, and preloading, performance improved substantially: repeated requests for cached pages were served in under 1 second, similar to a standard proxy; first-time translation latency was reduced by limiting the content sent to the LLM and optimizing request handling; and preloading was used to generate the translated options of a page in advance, before the user selected a specific style.

From a user experience perspective, the style transformations were generally accurate and expressive. On simple, static text pages, the transformed output preserved the semantic meaning while adjusting the tone and structure effectively. However, on more complex websites, the visuals and formatting are much more variable. Sometimes, the translated page loads with similar formatting to the original, while other times the CSS has some errors that result in the page losing its visual features while maintaining the translated text.

Reflection

The biggest challenge by far was performance. Because our project fundamentally depends on sending large portions of webpage text to an LLM, we were immediately constrained by high API latency and large request payload sizes. A significant portion of our time was spent profiling, experimenting with different HTML parsing/content extraction methods, and implementing caching and preloading to make the proxy faster.

The main additional functionality we would add would be to extend it to HTTPS websites. For the time being, we wanted to iron out HTTP functionality, so we spent the bulk of our time improving performance for HTTP webpages. As such, we didn't get to dedicate as much time to implementing HTTPS support as we would've liked. If the project scope allowed, adding a safe, standard method for HTTPS rewriting would be helpful.